Part D:
So, after researching this further and looking into the examples mentioned here is what I found. Firstly, Python (numpy) and R differ in terms of how they handle degrees of freedom when calculating standard deviation and variance. In Python it uses the population variance due to it using the default of ddof = 0 or delta degrees of freedom equal to 0. However, R uses sample variance which is better and less biased when calculating statistics by using ddof = 1 or that the samples is n - 1. Secondly, in Python nan values are skipped by default when calculating statistics (like the mean and sum) whereas R explicitly makes you decide how to deal with them before it will perform the calculation. Finally, R indexes based on 1 with inclusive intervals meaning that [1,5] is 1 - 5 and includes the endpoints where as Python is [1-5) but won't include 5 meaning that if you aren't expecting it, it can lead to an off by one error. Overall, most of the time R is either more accurate by default or requires you to be more specific before giving a result, whereas python gives a quick result but a result that may be less accurate due to making assumptions.